

■ OOP Mini Project Lab — Payroll Management System

■ Objective

Build a Payroll Management System using Python OOP concepts including Classes, Objects, Encapsulation, Inheritance, and Polymorphism.

■■ Suggested Time Breakdown

Task	Time
Understand Problem	15 mins
Create Classes	20 mins
Inheritance & Polymorphism	30 mins
CLI Interaction	25 mins
Testing	20 mins
Bonus Features	10 mins

Step 1: Create Base Employee Class

```
class Employee:

    def __init__(self, emp_id, name):
        self.emp_id = emp_id
        self.name = name
```

Step 2: Add Encapsulation

```
self.__salary = 0
```

Step 3: Add Common Methods

```
def calculate_salary(self):
    return 0
```

Step 4: Create FullTimeEmployee

```
class FullTimeEmployee(Employee):

    def __init__(self, emp_id, name, monthly_salary):
        super().__init__(emp_id, name)
        self.monthly_salary = monthly_salary
```

Step 5: Override Salary Method

```
def calculate_salary(self):
```

```
return self.monthly_salary
```

Step 6: Create Intern Class

```
class Intern(Employee):  
  
    def calculate_salary(self):  
        return self.stipend
```

Step 7: Create Contractor Class

```
class Contractor(Employee):  
  
    def calculate_salary(self):  
        return self.hourly_rate * self.hours_worked
```

Step 8: Store Employees

```
employees = []
```

Step 9: Add Employees

```
employees.append(e1)
```

Step 10: Display Payroll

```
for emp in employees:  
    print(emp.calculate_salary())
```

Step 11: Create Menu

```
while True:  
    print("1. Add Employee")
```

Step 12: Add User Input

```
choice = int(input("Enter choice: "))
```

Step 13: Handle Employee Types

```
if emp_type == 1:  
    # Full Time
```

Step 14: Total Payroll

```
total += emp.calculate_salary()
```

■ Bonus Features

- Add department attribute

- Save data into CSV
- Attendance-based salary
- Manager class with bonus
- Build GUI using Tkinter

■ Common CLI Instructions

- Develop terminal-based application
- Use minimum 3 classes
- Use inheritance and polymorphism
- Take dynamic user input
- Handle invalid inputs
- Display clean outputs

■ Submission Requirements

- Python source file
- Screenshots
- Project explanation
- Mention OOP concepts used